

Compile-time Object-Relational Mapping

библиотека на C++

Дипломна работа

Алекс Цветанов, фак. № 121222225

Научен ръководител: доц. д-р инж. Галя Павлова

Технически университет -- София
Факултет „Компютърни системи и технологии“
Компютърно и софтуерно инженерство

2026

Проблемът с runtime заявките

Традиционен подход:

- Заявките се описват като **низове**
- Валидиране едва при изпълнение
- Грешни имена на полета → runtime crash
- Несъвместими типове → late discovery
- Смяна на backend → **пълно пренаписване**

Изследователски въпрос:

Може ли компилаторът да поеме ролята на първи валидатор на заявките?

Runtime

Низ заявки

Грешка при
изпълнение

Late discovery

Compile-time

`constexpr` IR

Грешка при
компиляция

Early detection

Цел

Компилаторът поема ролята на първи валидатор на заявките.

Съществуващи решения и позиция на разработката

Решение	Подход	Ограничение
SOCI	runtime bind	низ-базирани за- явки
ODB sqlpp11	codegen EDSL	external compiler главно SQL сце- нарии
Hiberlite	ORM	само SQLite

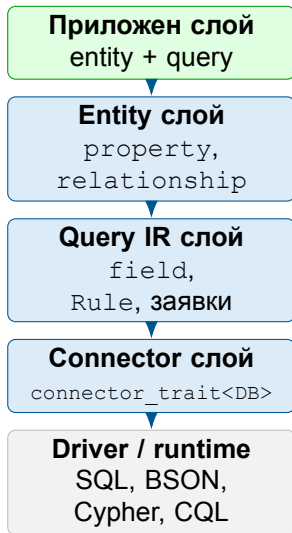
Цел на разработката

- `constexpr` query IR без codegen
- Connector contract и capability ограничения
- 7 backend-a и coroutine async слой

Основна теза

Един C++ модел може да бъде compile-time валидиран, multi-backend разширяем и practically useful при async натоварвания.

Слоеста архитектура на библиотеката



Ключови архитектурни принципа:

- Никакъв виртуален интерфейс trait-базиран, не наследствен contract
- Query IR е `constexpr` значима валидация при компилация
- Capability tags ограничават неподдържаните операции
- Смяна на backend = смяна само на типа на конектора

Централен инвариант

Приложният код не комуникира директно с драйвера — работи само с C++ типове.

Минимален compile-time пример

```
struct User {  
    orm::property<int, "id"> id;  
    orm::property<std::string, "name"> name;  
};  
  
static constexpr auto q =  
    orm::select(  
        orm::field<&User::id>,  
        orm::field<&User::name>  
    ).where(orm::field<&User::id>  
        == orm::Placeholder<int>{});
```

- `property<T, "name">` носи тип и compile-time име на колона
- Заявката се изгражда като `constexpr IR`, а не като низ
- Грешно поле или неподдържана операция се открива рано

Ключова идея

Компиляторът поема ролята на първи валидатор на заявката.

Connector trait и Sarability механизъм

Минимален connector contract:

```
template<>
struct orm::connector_trait<MyDB> {
    template<typename T> struct wire_type;
    using cursor_type = ...;

    using supports_transactions = true_type;

    static auto execute(
        MyDB& db, auto query,
        auto&&... params);
};
```

- `is_connector<DB>` валидира интерфейса при компилация
- Сарability таговете ограничават неподдържани операции
- Смяната на backend е смяна на типа, а не на query API

Демонстрирани backend-и

SQLite, PostgreSQL, MySQL,
MongoDB, Redis, Neo4j, Cassandra

Multi-backend материализация

Един Query IR → различни backend-и:

Backend	Материализация
SQLite	WHERE id = ?
PostgreSQL	WHERE id = \$1
MySQL	WHERE id = ?
MongoDB	BSON filter
Redis	GET/SET/DEL
Neo4j	Cypher MATCH...WHERE
Cassandra	CQL SELECT...FROM

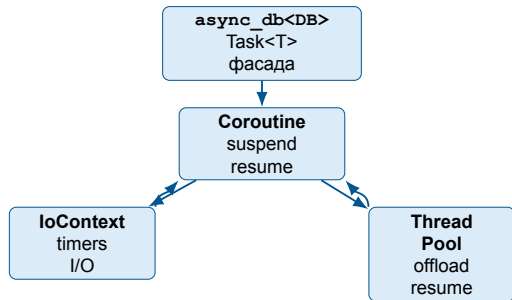
Абстракция на връзките:

embed	MongoDB вложен документ
reference	SQL foreign key / JOIN
	Redis: key prefix
	Neo4j: edge

Принцип

C++ моделът е единственият *логически* носител на схемата. Backend-ът определя физическата материализация.

Асинхронна архитектура на изпълнението



- При waiting фаза worker нишката не блокира, а корутината се suspend-ва
- Синхронен connector може да се повдигне чрез offload към pool
- Native async path остава възможен чрез awaitable слой
- Пулове и транзакции запазват RAII семантиката

Ключов извод

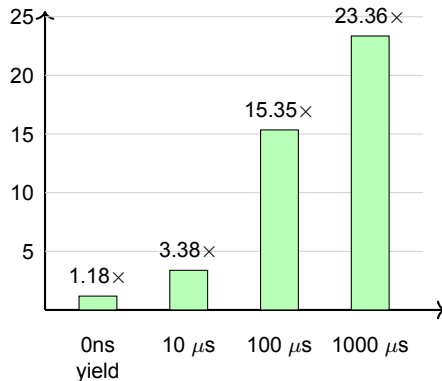
Async слойът е втора архитектурна ос на библиотеката, а не ad-hoc wrapper.

Доказателствена база

271 теста — 100% преминаване
Compile-time + unit + integration + live
backend проверки

- Query IR, capability ограничения, миграции и нишкова безопасност
- Async инфраструктура: `Task<T>`, `ThreadPool`, `IoContext`, `async_db<DB>`
- Benchmark suite в `bench_async.cpp` за synchronous vs async throughput

ускорение async/sync



4 нишки / 100 задачи. При 0ns ефектът е малък; при доминираща waiting

фаза async печели силно.

Основни резултати и извод

- 1 Compile-time ORM модел в стандартен C++23 без external codegen
- 2 Един C++ entity модел за 7 backend-a
- 3 Формален connector contract и coroutability-базирана разширяемост
- 4 Async слой с измерима полза при I/O-dominated натоварвания
- 5 Многостепенна верификация: 271 теста + live backend + benchmark evidence

Основен извод

Между runtime ORM и driver-only достъп съществува жизнеспособна архитектурна позиция: compile-time ORM слой с формални contracts и coroutine-базирано изпълнение.

Бъдещо развитие:

- C++26 static reflection
- native async конектори
- по-пълна migration автоматизация

Благодаря за вниманието!

Въпроси?